Cloud and Mobile Computing

Lab 2 Report

Distributed Consistency and Consensus in the Cloud

Student Name:	Bassel Adel
Group:	SE3
Lab Title:	Distributed Consistency and Consensus in the Cloud

Learning Objectives

• Simulate eventual consistency with Redis replicas.

• Create a network partition and observe replication behavior.

• Experiment with Raft consensus using etcd.

• Observe leader election and failover in a Raft cluster.

Environment Setup

The lab environment was set up using Docker Compose with three containers: redis-node1 (master), redis-node2 (replica), and etcd (consensus store). All containers were launched successfully using docker-compose up.

Task 1: Redis Replication and CAP Theorem

Redis was configured in a master-replica setup where redis-node1 acts as the master and redis-node2 automatically replicates all data from it. This demonstrates eventual consistency — a key concept in the CAP theorem.

Step 1 & 2: Write to Master, Read from Replica

A key was written to redis-node1 and immediately read from redis-node2 to verify replication:

```
docker exec -it redis-node1 redis-cli set mykey "hello from node1" → OK
docker exec -it redis-node2 redis-cli get mykey → "hello from node1"
```

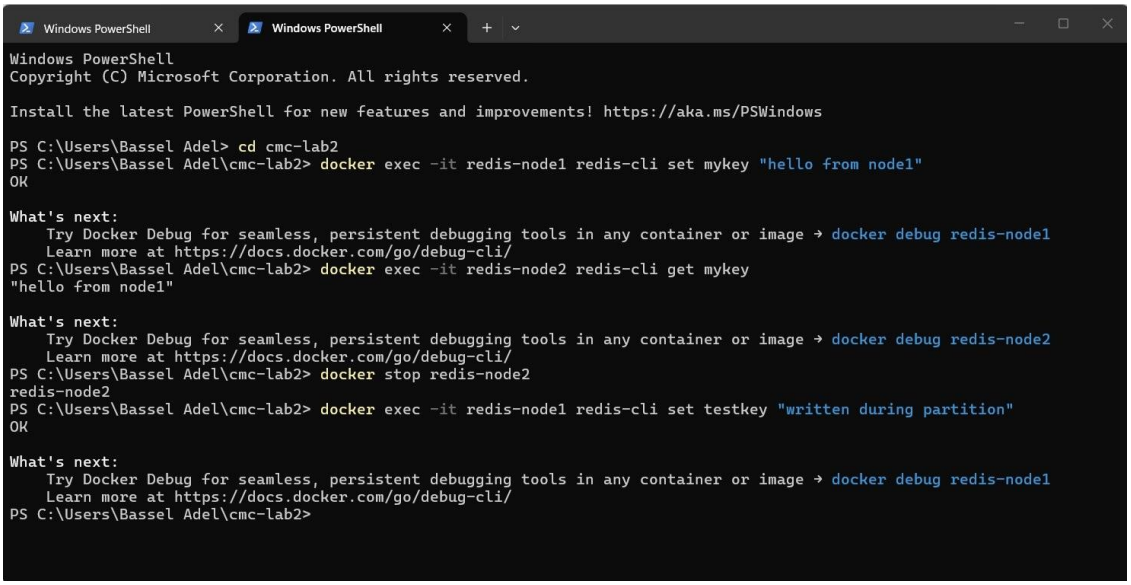

Figure 1: Writing to redis-node1 and reading from redis-node2 — replication confirmed

Step 3 & 4: Simulate Network Partition

redis-node2 was stopped to simulate a network partition. A new key was then written to redis-node1 while the replica was down:

```
docker stop redis-node2
docker exec -it redis-node1 redis-cli set testkey "written during partition" → OK
```

redis-node1 continued accepting writes even without the replica — demonstrating the Availability (A) property of the CAP theorem. Redis prioritizes Availability over Consistency during a partition, making it an AP system.

Observation	Result	CAP Implication
Read from replica after write	"hello from node1" returned	Replication works (Consistency)
Write during partition	OK — write succeeded	Availability maintained
Replica rejoins	Data syncs automatically	Eventual Consistency

Task 2: Raft Consensus with etcd

etcd is a distributed key-value store that uses the Raft consensus algorithm to ensure strong consistency. In this task, key-value operations were performed and leader election was observed by stopping and restarting the etcd container.

Step 1 & 2: Write and Read Key-Value

A key-value pair was stored in etcd and retrieved successfully:

```
docker exec -it etcd etcdctl put foo bar → OK
docker exec -it etcd etcdctl get foo → foo / bar
```

Step 3: Check Leader Status (Before Restart)

The endpoint status was checked to confirm the current leader:

```
docker exec -it etcd etcdctl endpoint status
127.0.0.1:2379, 8e9e05c52164694d, 3.5.12, 20 kB, true, false, 2, 5, 5,
```

The field `true` confirms this node is the current leader at `term 2` with committed index 5.

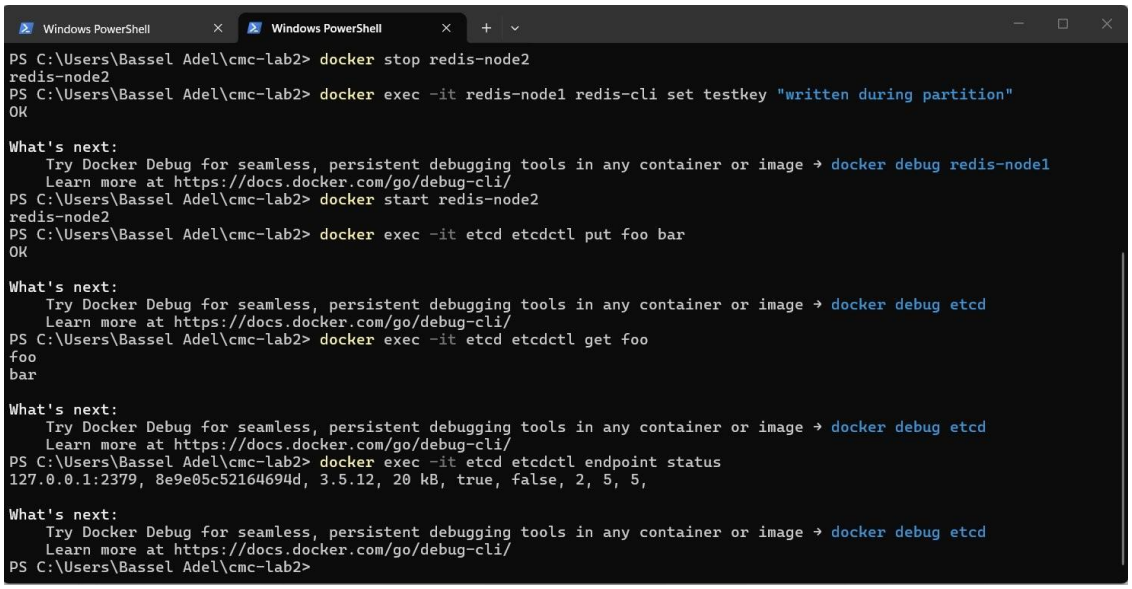

Figure 2: etcd put/get and endpoint status showing leader at term 2

Step 4: Leader Re-election After Restart

The etcd container was stopped and restarted to trigger a new Raft election:

```
docker stop etcd
docker start etcd
docker exec -it etcd etcdctl endpoint status
127.0.0.1:2379, 8e9e05c52164694d, 3.5.12, 20 kB, true, false, 3, 7, 7,
```

```
What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug etcd
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Bassel Adel\cmc-lab2> docker stop etcd
etcd
PS C:\Users\Bassel Adel\cmc-lab2> docker start etcd
etcd
PS C:\Users\Bassel Adel\cmc-lab2> docker exec -it etcd etcdctl endpoint status
127.0.0.1:2379, 8e9e05c52164694d, 3.5.12, 20 kB, true, false, 3, 7, 7,

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug etcd
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\Bassel Adel\cmc-lab2>
```

Figure 3: After restart — term increased from 2 to 3, confirming new leader election

Field	Before Restart	After Restart	Meaning
Raft Term	2	3	New election occurred
Committed Index	5	7	New entries committed
Is Leader	true	true	Node re-elected as leader

Discussion and Analysis

1. CAP Theorem and Redis

Redis demonstrates the AP (Available + Partition Tolerant) side of the CAP theorem. During the simulated network partition, redis-node1 continued accepting writes without waiting for redis-node2 to acknowledge them. This means Redis sacrifices strong consistency in favor of availability. When the replica rejoined, it eventually caught up — demonstrating eventual consistency rather than strong consistency.

2. Raft Consensus and etcd

etcd uses the Raft protocol to guarantee strong consistency (CP system). When the etcd container was restarted, Raft automatically triggered a new election, incrementing the term from 2 to 3. This ensures that at any point in time, only one leader exists and all writes go through it, preventing split-brain scenarios. Unlike Redis, etcd would refuse writes if a quorum cannot be reached.

3. Redis vs etcd — When to Use Each

Feature	Redis (AP)	etcd (CP)
Consistency	Eventual	Strong (linearizable)
Availability	High — always accepts writes	Lower — needs quorum
Use Case	Caching, sessions, leaderboards	Config store, service discovery
Partition Behavior	Continues writing	Refuses writes without quorum
Consensus Protocol	None (async replication)	Raft

Conclusion

This lab demonstrated two fundamental approaches to consistency in distributed systems. Redis with async replication showed how an AP system maintains high availability at the cost of temporary inconsistency during partitions. etcd with Raft consensus showed how a CP system guarantees strong consistency through leader election and quorum-based writes. Understanding these trade-offs is essential for designing reliable cloud-native applications.